23BEC0503

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRONICS ENGINEERING

Fall Semester 2025-2026

BECE406E – FPGA Based System Design

TASK 1 – Design and Implementation of Adders and Multipliers

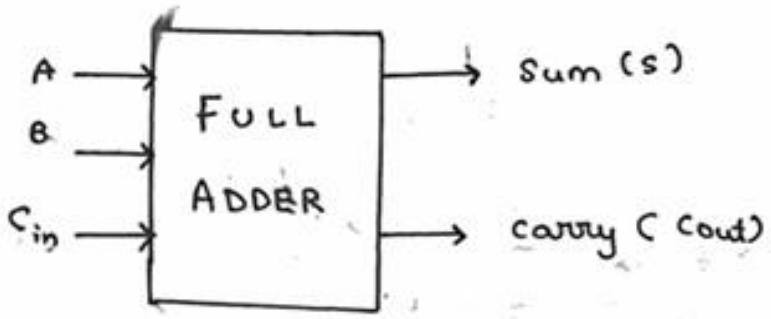
SANJAY ELAVARASAN KARTHIKEYAN

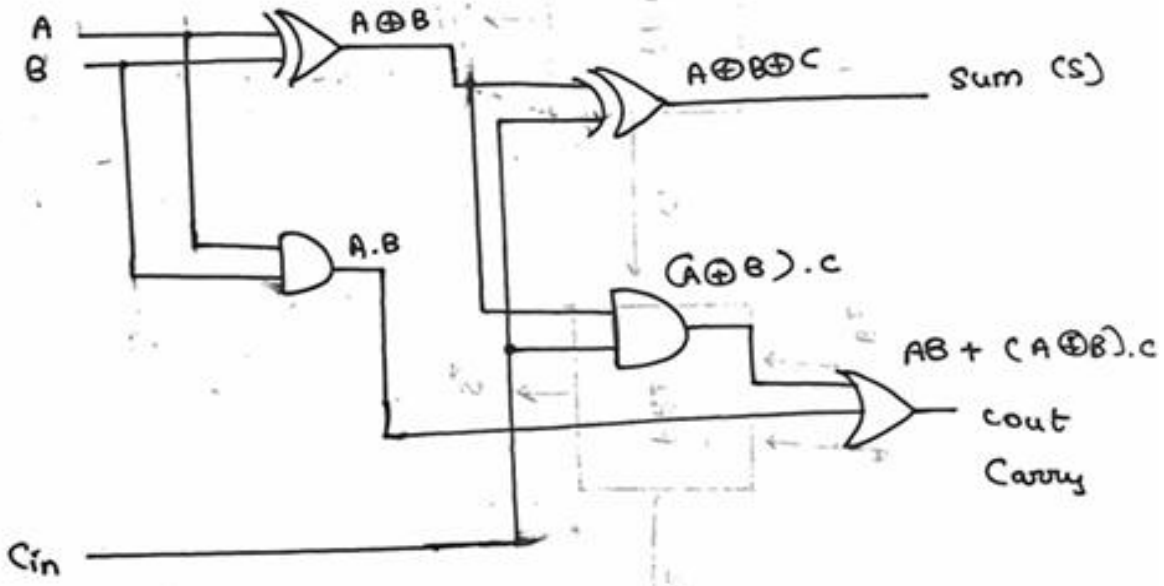
23BEC0503

23BEC0503

i) Design and verify Full adder using gate level/ structural modelling

Design :





Sum = A ⊕ B ⊕ Cin ;

Carry = AB + BCin + ACin ;

23BEC0503

Verilog Modelling (Gate-Level / Structural Modeling):

```
module full_adder (
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
);

    wire w1, w2, w3;

    // Sum = A ⊕ B ⊕ Cin
    xor (w1, A, B);
    xor (Sum, w1, Cin);

    // Carry = AB + BCin + ACin
    and (w2, A, B);
    and (w3, w1, Cin);
    or  (Cout, w2, w3);

endmodule
```

full_adder.v | full_adder_test.v

23BEC0503

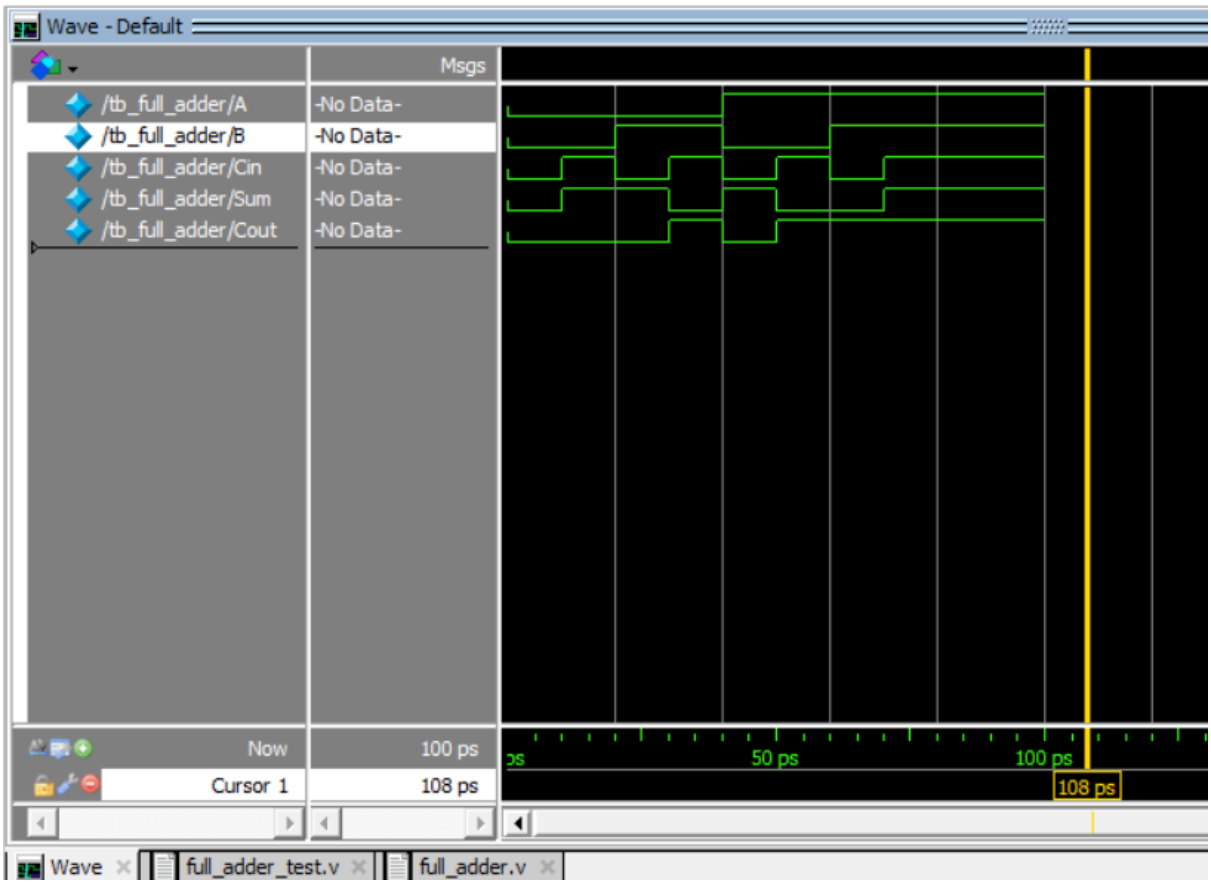
Testbench for Verification

```
Ln#
1   module tb_full_adder;
2
3       reg A, B, Cin;
4       wire Sum, Cout;
5
6       full_adder DUT (A,B,Cin,Sum,Cout);
7
8       initial begin
9           A=0; B=0; Cin=0; #10;
10          A=0; B=0; Cin=1; #10;
11          A=0; B=1; Cin=0; #10;
12          A=0; B=1; Cin=1; #10;
13          A=1; B=0; Cin=0; #10;
14          A=1; B=0; Cin=1; #10;
15          A=1; B=1; Cin=0; #10;
16          A=1; B=1; Cin=1; #10;
17
18      end
19
20  endmodule
21
22
```

full_adder.v | full_adder_test.v

23BEC0503

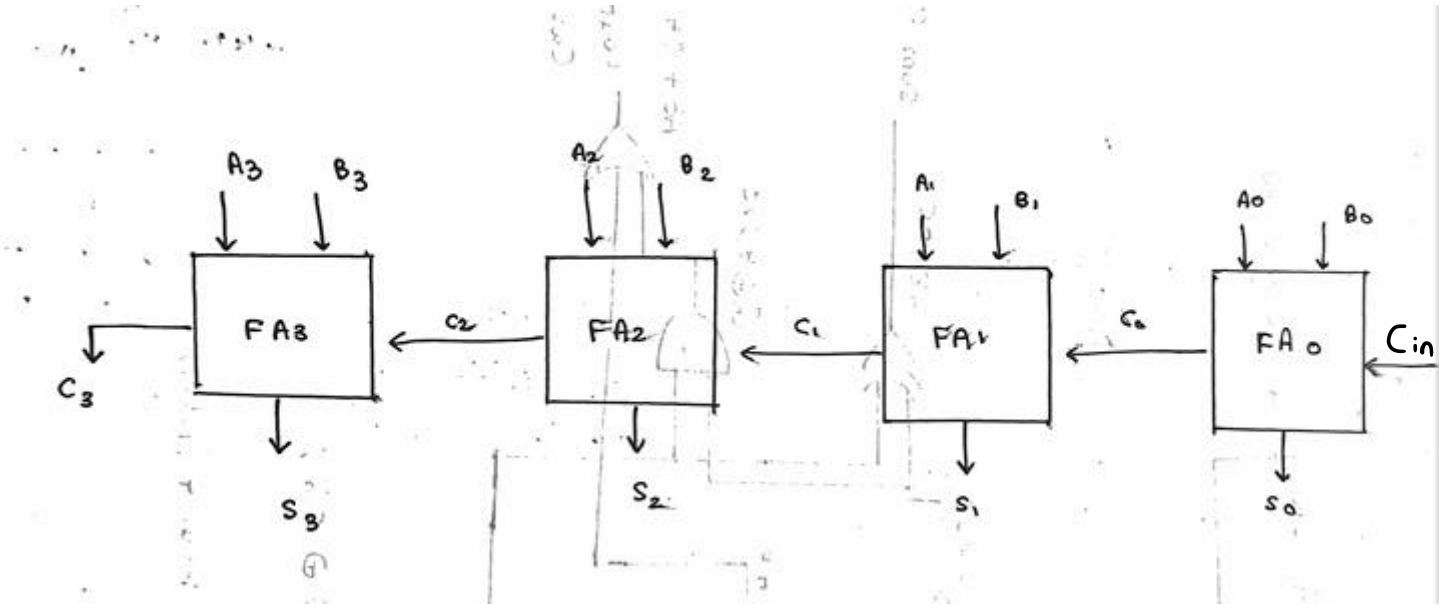
Output Waveform :



23BEC0503

ii) Using the designed full adder, design a 4-bit Ripple Carry Adder (RCA) and verify the same using testbench.

Design :



Verilog Modeling :

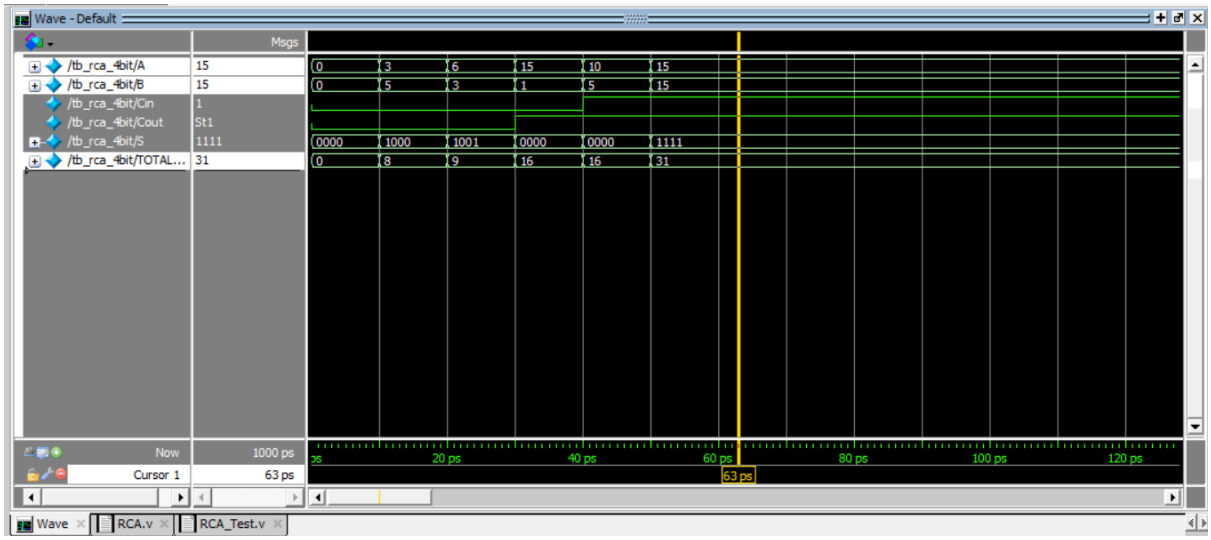
```
//Structural Modeling - RCA
module rca_4bit (
    input  [3:0] A,
    input  [3:0] B,
    input        Cin,
    output [3:0] S,
    output       Cout,
    output [4:0] TOTAL_SUM   // overall sum
);

    wire C1, C2, C3;

    full_adder FA0 (A[0], B[0], Cin, S[0], C1); //Instantiation of Full adder
    full_adder FA1 (A[1], B[1], C1,  S[1], C2);
    full_adder FA2 (A[2], B[2], C2,  S[2], C3);
    full_adder FA3 (A[3], B[3], C3,  S[3], Cout);

    // Combine carry and sum
    assign TOTAL_SUM = {Cout, S};

endmodule



//Gate-level - Modeling(Full adder)
module full_adder (
    input  A,
    input  B,
    input  Cin,
    output Sum,
    output Cout
);

    wire w1, w2, w3;

    xor (w1, A, B);
    xor (Sum, w1, Cin);

    and (w2, A, B);
    and (w3, w1, Cin);
    or  (Cout, w2, w3);

endmodule
```

23BEC0503

Testbench for Verification :

```
Ln#
1   module tb_rca_4bit;
2
3       reg  [3:0] A, B;
4       reg        Cin;
5       wire [3:0] S;
6       wire       Cout;
7       wire [4:0] TOTAL_SUM;
8
9       // Instantiate DUT
10      rca_4bit DUT (A,B,Cin,S,Cout,TOTAL_SUM);
11
12      initial begin
13          $monitor("Time=%0t | A=%d B=%d Cin=%b -- TOTAL_SUM=%d (Cout=%b, S=%b)",
14                   $time, A, B, Cin, TOTAL_SUM, Cout, S);
15
16          A = 4'd0;  B = 4'd0;  Cin = 0; #10;
17          A = 4'd3;  B = 4'd5;  Cin = 0; #10;
18          A = 4'd6;  B = 4'd3;  Cin = 0; #10;
19          A = 4'd15; B = 4'd1;  Cin = 0; #10;
20          A = 4'd10; B = 4'd5;  Cin = 1; #10;
21          A = 4'd15; B = 4'd15; Cin = 1; #10;
22
23
24      end
25
26  endmodule
27
```

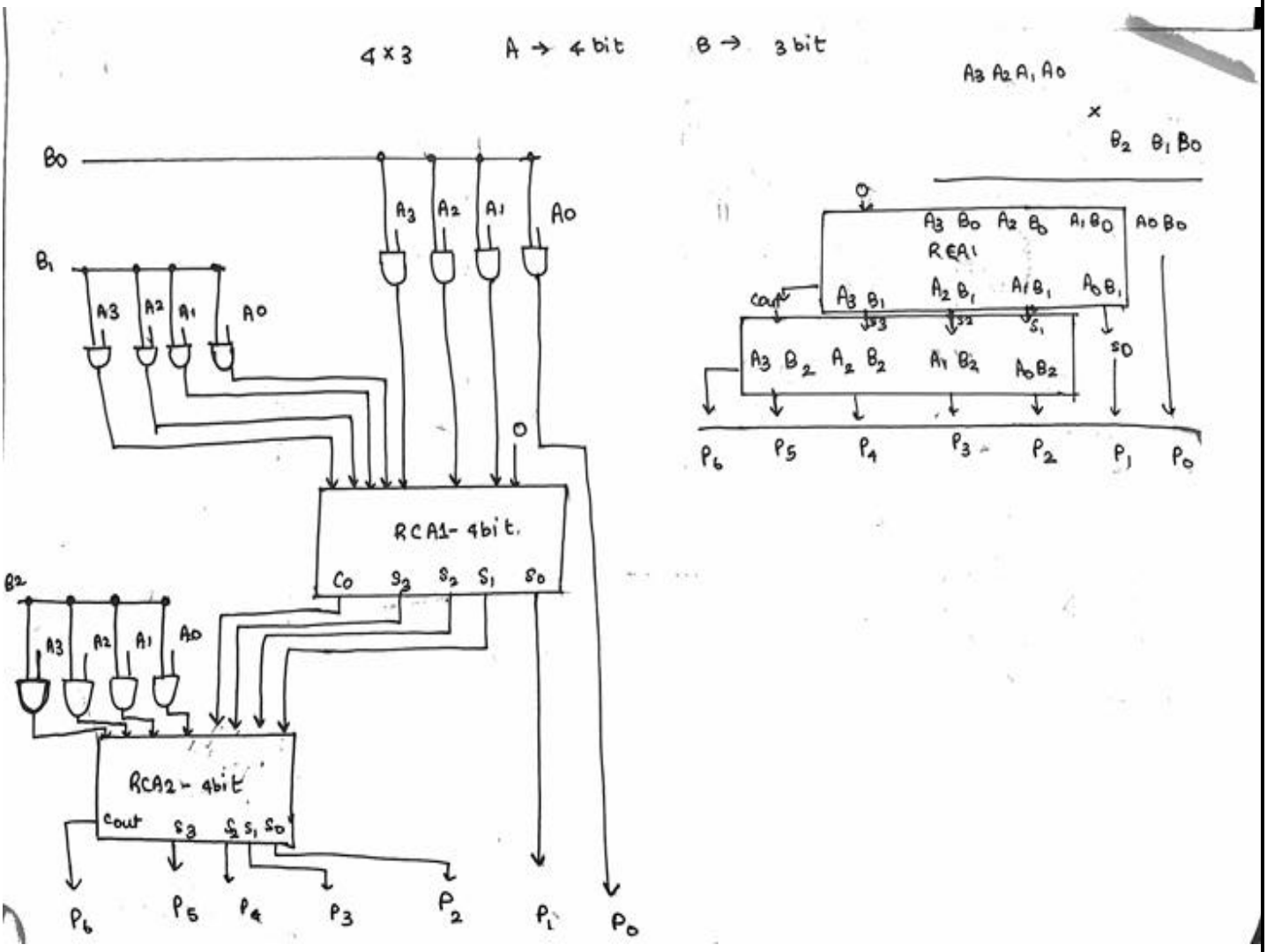
Output Waveform :




```
VSIM 7> run
# Time=0 | A=0000 B=0000 Cin=0 --> Sum=0000 Cout=0
# Time=10 | A=0011 B=0101 Cin=0 --> Sum=1000 Cout=0
# Time=20 | A=0110 B=0011 Cin=0 --> Sum=1001 Cout=0
# Time=30 | A=1111 B=0001 Cin=0 --> Sum=0000 Cout=1
# Time=40 | A=1010 B=0101 Cin=1 --> Sum=0000 Cout=1
# Time=50 | A=1111 B=1111 Cin=1 --> Sum=1111 Cout=1
VSIM 8> quit -sim
# End time: 16:58:44 on Jan 09,2026, Elapsed time: 0:02:30
```

iii) Using the RCA designed in question ii) and few logic gates, design a 4x3 multiplier and verify the same using testbench.

Design :



23BEC0503

Verilog Modeling :

```
Ln#
module multi_4x3(A,B,P);

input [3:0]A;
input [2:0]B;


output [6:0]P;

wire pp00,pp10,pp20,pp30,pp01,pp11,pp21,pp31,pp02,pp12,pp22,pp32,s0,s1,s2,s3,c0,c1;
//GENERATE PARTIAL PRODUCTS
assign pp00 = A[0]& B[0];
assign pp10 = A[1]& B[0];
assign pp20 = A[2]& B[0];
assign pp30 = A[3]& B[0];

assign pp01 = A[0]& B[1];
assign pp11 = A[1]& B[1];
assign pp21 = A[2]& B[1];
assign pp31 = A[3]& B[1];

assign pp02 = A[0]& B[2];
assign pp12 = A[1]& B[2];
assign pp22 = A[2]& B[2];
assign pp32 = A[3]& B[2];


//INSTANTIATE - RCA
RCA RCA1({1'b0,pp30,pp20,pp10},{pp31,pp21,pp11,pp01},1'b0,{s3,s2,s1,s0},c0);
RCA RCA2({c0,s3,s2,s1},{pp32,pp22,pp12,pp02},1'b0,{P[5],P[4],P[3],P[2]},c1);


assign P[0] = pp00;
assign P[1] = s0;
assign P[6] = c1;


endmodule
```

```
// STRUCTURAL MODELING RCA - RIPPLE CARRY ADDER
module RCA(A,B,cin,Sum,Cout);

input [3:0]A,B;
input cin;
output [3:0]Sum;
output Cout;

wire c0,c1,c2;
//INSTANTIATE FULL ADDER
full_adder FA1 (Sum[0],c0,A[0],B[0],cin);
full_adder FA2 (Sum[1],c1,A[1],B[1],c0);
full_adder FA3 (Sum[2],c2,A[2],B[2],c1);
full_adder FA4 (Sum[3],Cout,A[3],B[3],c2);


endmodule

//GATE LEVEL MODELING - FULL ADDER
module full_adder (Sum,Cout,A,B,Cin);
input A,B,Cin;
output Sum,Cout;

    wire w1, w2, w3;

    xor (w1, A, B);
    xor (Sum, w1, Cin);

    and (w2, A, B);
    and (w3, w1, Cin);
    or  (Cout, w2, w3);

endmodule
```

23BEC0503

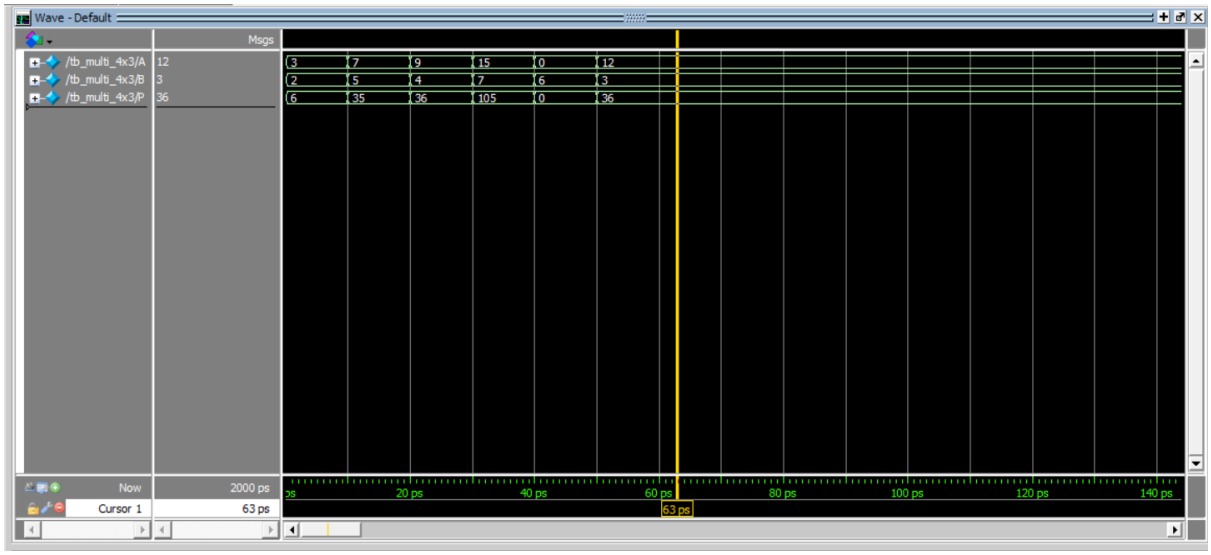
Testbench for Verification :

```
Ln#
 1  module tb_multi_4x3;
 2
 3  reg  [3:0] A;
 4  reg  [2:0] B;
 5  wire [6:0] P;
 6
 7  multi_4x3 DUT (
 8      .A(A),
 9      .B(B),
10      .P(P)
11  );
12
13
14  initial begin
15      $display(" A   B   |  P");
16      $display("----------------");
17
18      A = 4'd3;  B = 3'd2;  #10; // 6
19      A = 4'd7;  B = 3'd5;  #10; // 35
20      A = 4'd9;  B = 3'd4;  #10; // 36
21      A = 4'd15; B = 3'd7;  #10; // 105
22      A = 4'd0;  B = 3'd6;  #10; // 0
23      A = 4'd12; B = 3'd3;  #10; // 36
24
25      $finish;
26  end
27
28  initial begin
29      $monitor("%d  %d = %d", A, B, P);
30  end
31
32  endmodule
33
34
```

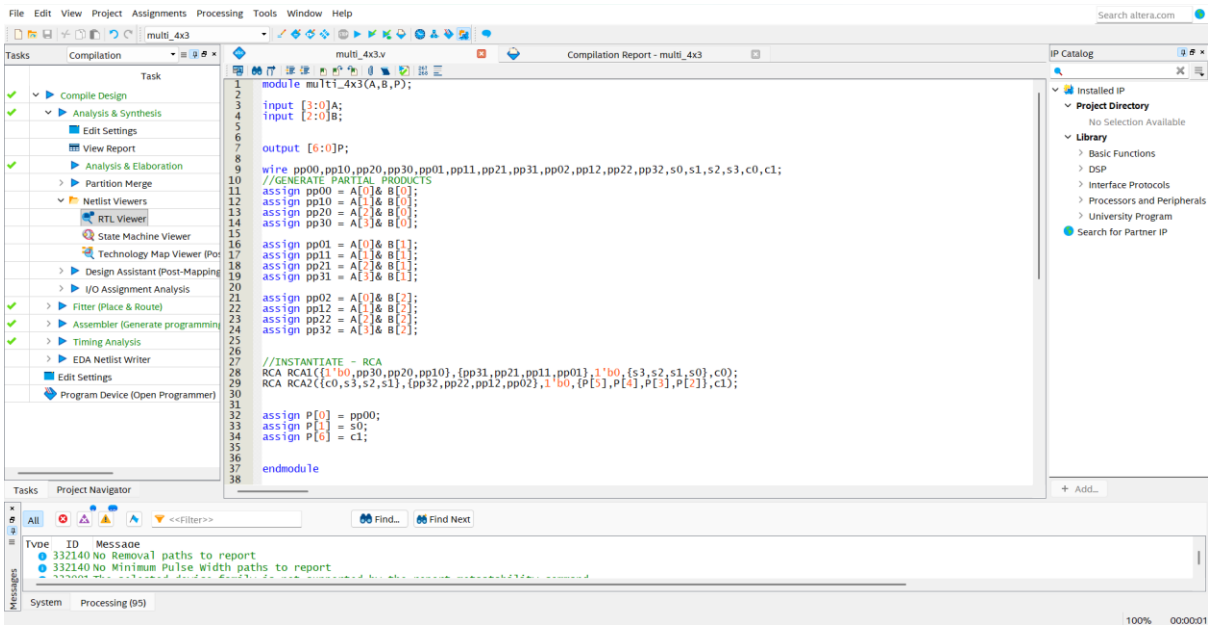
Output Waveform :

```
Transcript
sim:/tb_multi_4x3/A \
sim:/tb_multi_4x3/B \
sim:/tb_multi_4x3/P
VSIM 15> run
#  A   B   |  P
# ----------------
#  3   2 =   6
#  7   5 =  35
#  9   4 =  36
# 15   7 = 105
#  0   6 =   0
# 12   3 =  36
# ** Note: $finish    : C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/Multiplier_4x3_Test.v(25)
#    Time: 60 ps  Iteration: 0  Instance: /tb_multi_4x3
# 1
# Break in Module tb_multi_4x3 at C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/Multiplier_4x3_Test.v line 25
VSIM 16> run -continue
Processes   Transcript
```

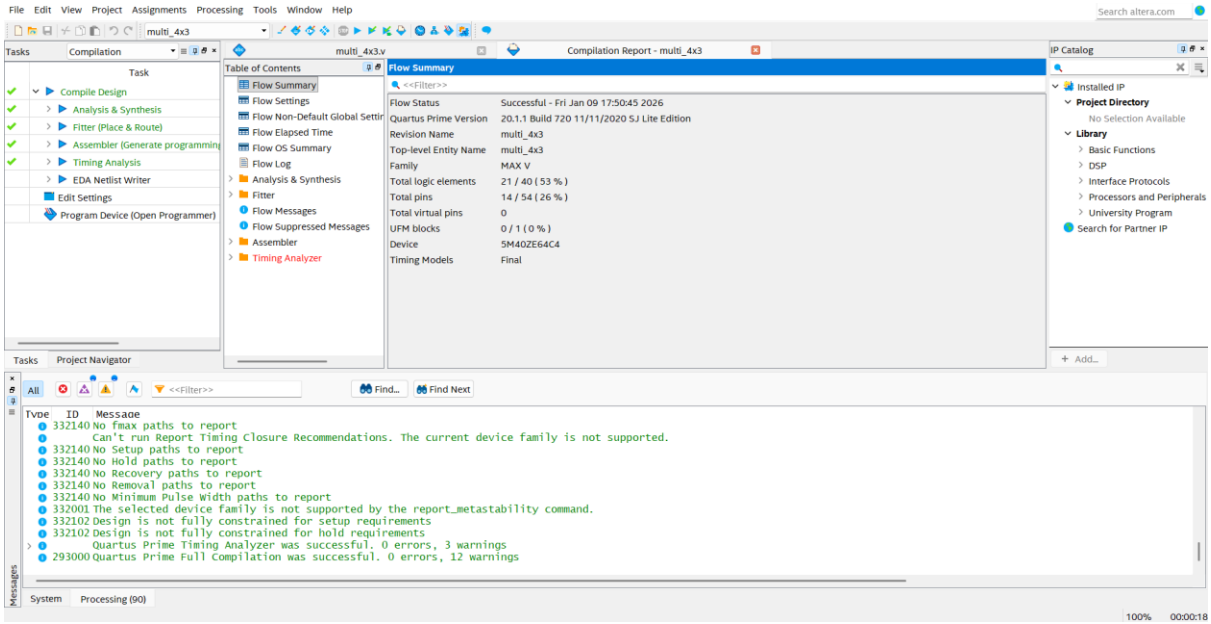
23BEC0503



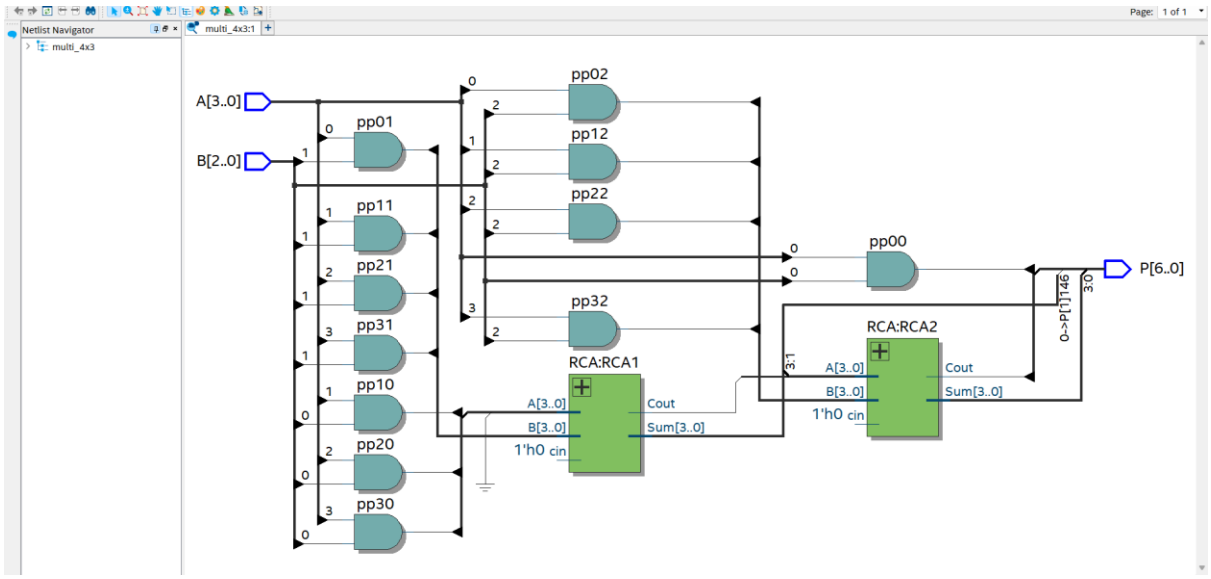
iv) Implement the 4x3 multiplier in DE2-115 Board using Intel Quartus Prime.



23BEC0503

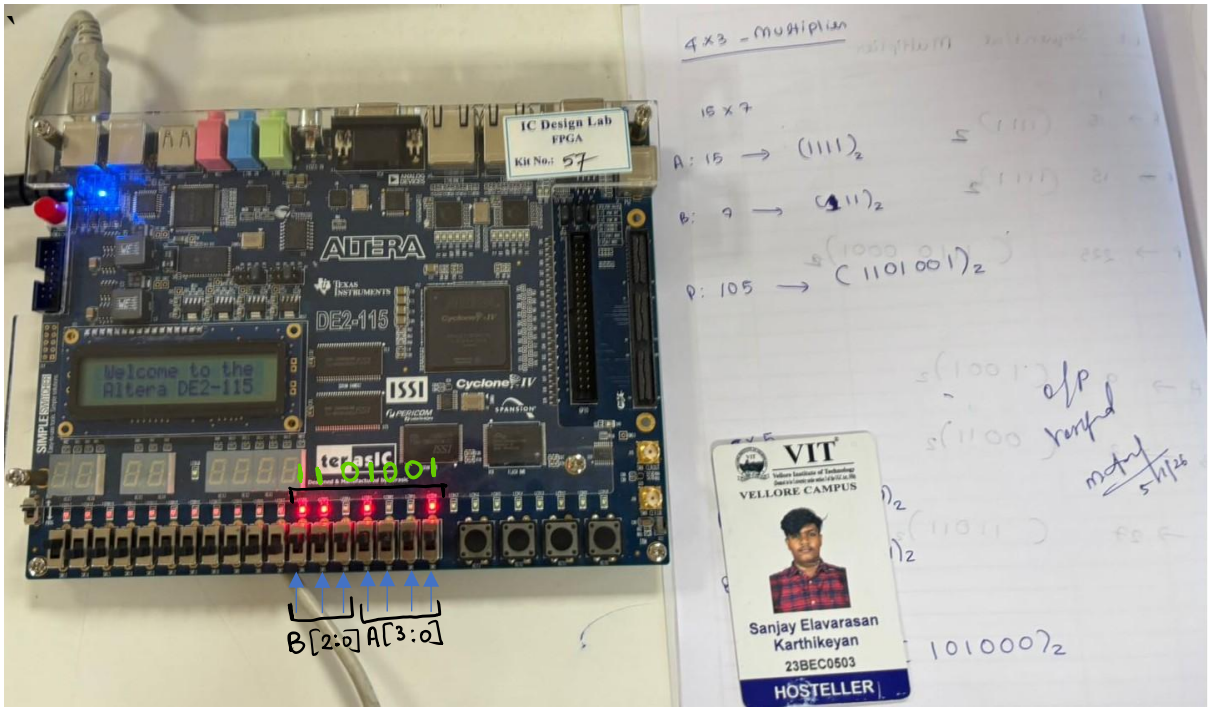


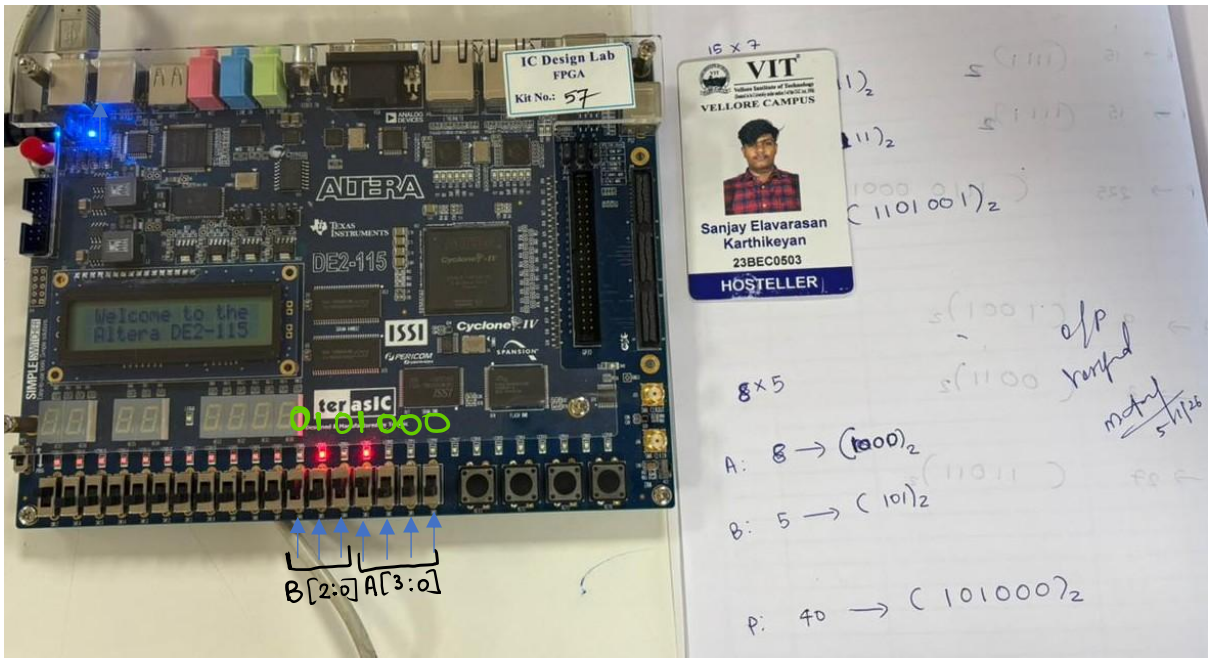
Gate Level - NetList



23BEC0503

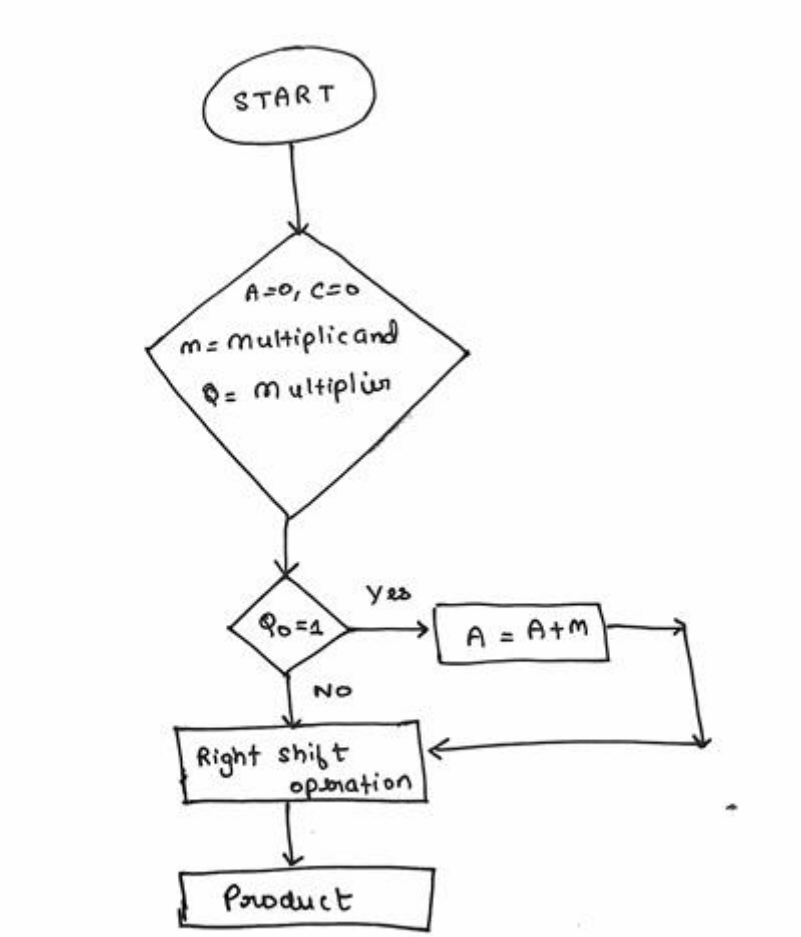
OUTPUT :





v) Attach Screenshots of code and outputs of 4-bit sequential multiplier.

Algorithm :



23BEC0503

Verilog Modeling :

```
Ln#
module Sequenital_4bit (
    input clk,
    input rst,
    input disp,
    input [3:0] A, // 4-bit multiplicand
    input [3:0] B, // 4-bit multiplier
    output reg [7:0] P, // 8-bit final product
    output reg Wait,
    output reg done
);

    reg [3:0] multiplicand;
    reg [8:0] acc;      // 9-bit Accumulator (Carry + 4-bit partial sum + 4-bit multiplier)
    reg [3:0] multiplier;
    reg [2:0] count;    // 3 bits enough to count to 4
    wire [4:0] x, y;

    // RCA instances for addition logic
    Adder_4bit Adder1(A, acc[7:4], x[3:0], x[4]);
    Adder_4bit Adder2(4'b0, acc[7:4], y[3:0], y[4]);

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            multiplicand <= 0;
            multiplier <= 0;
            acc <= 0;
            P <= 0;
            count <= 0;
            done <= 0;
            Wait <= 0;
        end
        else if (disp && !Wait) begin
            multiplicand <= A;
            multiplier <= B;
            count <= 4; // Set for 4 cycles
            done <= 0;
            Wait <= 1;
        end
        else if (Wait) begin
            if (multiplier[0] == 1)
                acc[8:4] = x;
            else
                acc[8:4] = y;

            multiplier = multiplier >> 1;
            acc = acc >> 1;
            count = count - 1;

            if (count == 0) begin
                P <= acc[7:0];
                done <= 1;
                Wait <= 0;
            end
        end
    end

    always @(posedge disp) begin
        acc <= 0;
        P <= 0;
    end




endmodule

// 4-bit Ripple Carry Adder Module
module Adder_4bit (
    input [3:0] A,
    input [3:0] B,
    output [3:0] Sum,
    output Cout
);
    wire c1, c2, c3;

    FullAdder fa0 (A[0], B[0], 1'b0, Sum[0], c1);
    FullAdder fa1 (A[1], B[1], c1,   Sum[1], c2);
    FullAdder fa2 (A[2], B[2], c2,   Sum[2], c3);
    FullAdder fa3 (A[3], B[3], c3,   Sum[3], Cout);

endmodule


// Full Adder
module FullAdder (
    input A, B, Cin,
    output Sum, Cout
);
    wire w1, w2, w3;

    xor (w1, A, B);
    xor (Sum, w1, Cin);

    and (w2, A, B);
    and (w3, w1, Cin);
    or  (Cout, w2, w3);
endmodule
```

23BEC0503

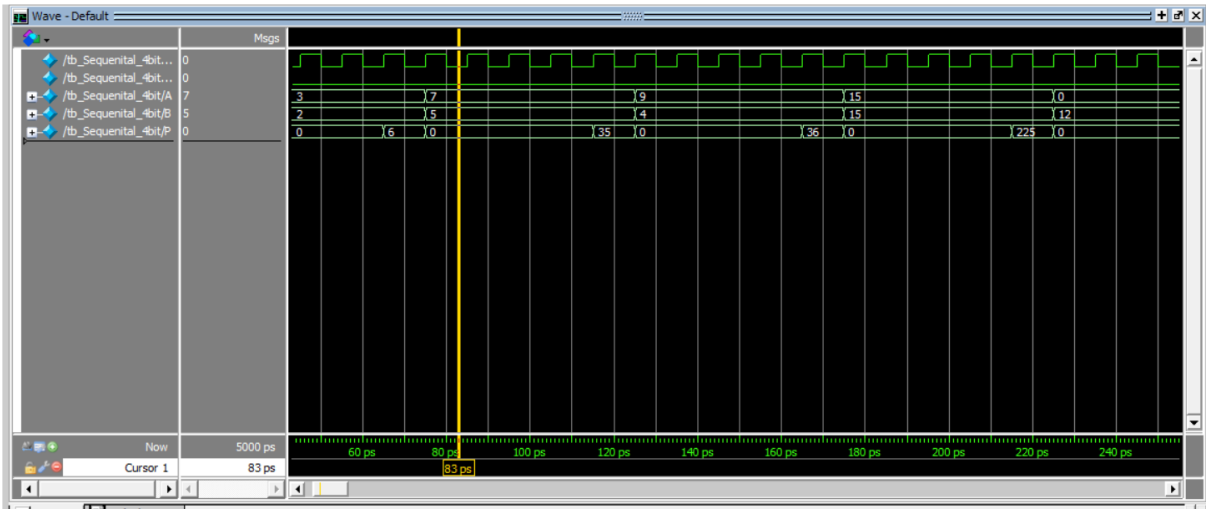
Testbench for Verification :

```
Ln#
module tb_Sequenital_4bit;

    reg clk;
    reg rst;
    reg disp;
    reg [3:0] A;
    reg [3:0] B;

    wire [7:0] P;
    wire Wait;
    wire done;

    // Instantiate DUT
    Sequenital_4bit DUT (clk,rst,disp,A,B,P,Wait,done);

    // Clock generation: 10 ns period
    always #5 clk = ~clk;


    initial begin
        // Initial values
        clk  = 0;
        rst  = 1;
        disp = 0;
        A    = 0;
        B    = 0;

        // Reset
        #20;
        rst = 0;

        A = 4'd3;
        B = 4'd2;
        disp = 1;
        #10;
        disp = 0;

        wait(done);
        #10;
        $display("A=%d B=%d -> Z=%d (Expected 6)", A, B, P);
 ..
53
        A = 4'd9;
        B = 4'd4;
        disp = 1;
        #10;
        disp = 0;

        wait(done);
        #10;
        $display("A=%d B=%d -> Z=%d (Expected 36)", A, B, P);


        A = 4'd15;
        B = 4'd15;
        disp = 1;
        #10;
        disp = 0;

        wait(done);
        #10;
        $display("A=%d B=%d -> Z=%d (Expected 225)", A, B, P);


        A = 4'd0;
        B = 4'd12;
        disp = 1;
        #10;
        disp = 0;

        wait(done);
        #10;
        $display("A=%d B=%d -> Z=%d (Expected 0)", A, B, P);

        #20;

    end

endmodule
```

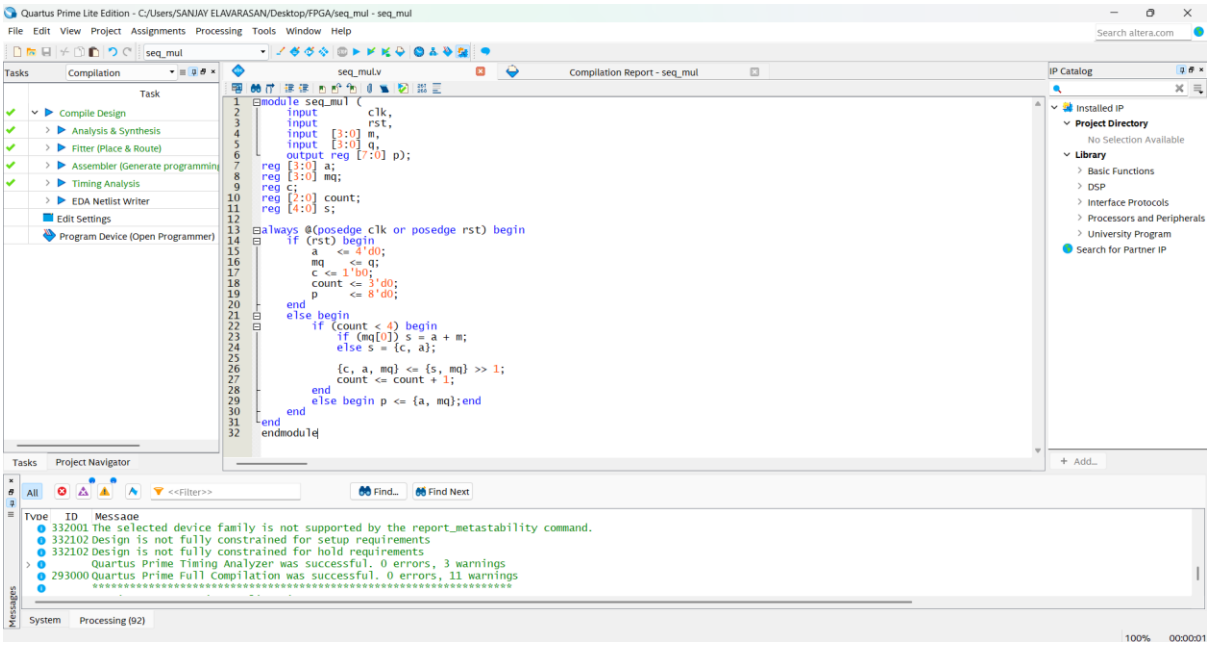
Output Waveform :

```
Wave | Multiplier.v | Multiplier_test.v
Transcript
VSIM 53> run
# A= 3 B= 2 -> Z=  6 (Expected 6)
# A= 7 B= 5 -> Z= 35 (Expected 35)
# A= 9 B= 4 -> Z= 36 (Expected 36)
# A=15 B=15 -> Z=225 (Expected 225)
# A= 0 B=12 -> Z=  0 (Expected 0)

VSIM 54>
Processes | Transcript
```

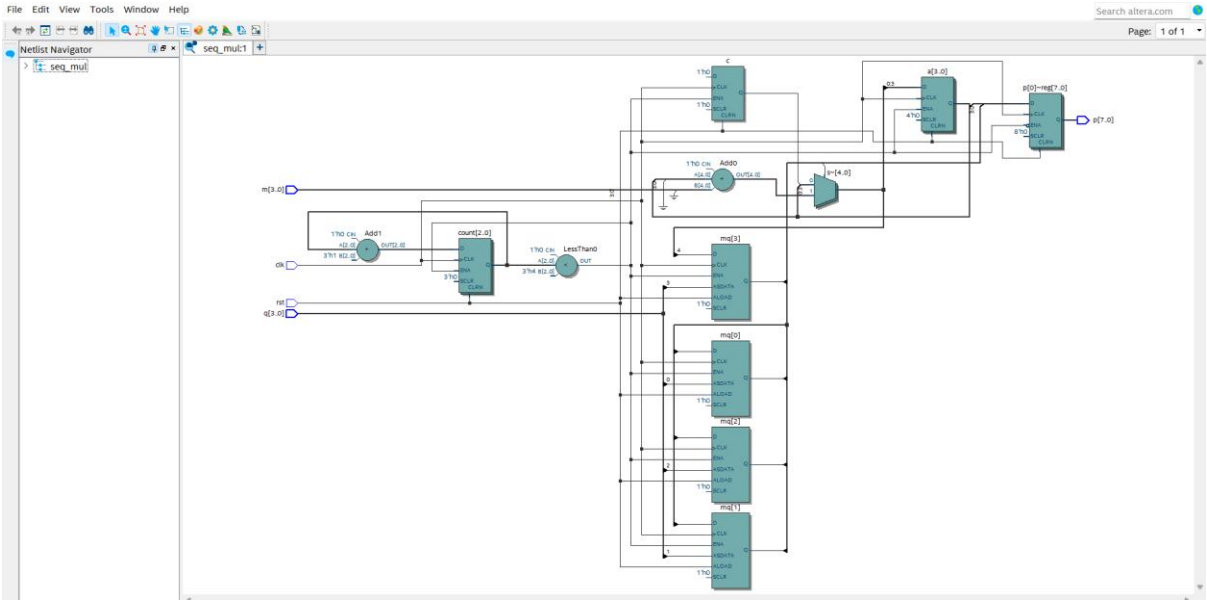
23BEC0503



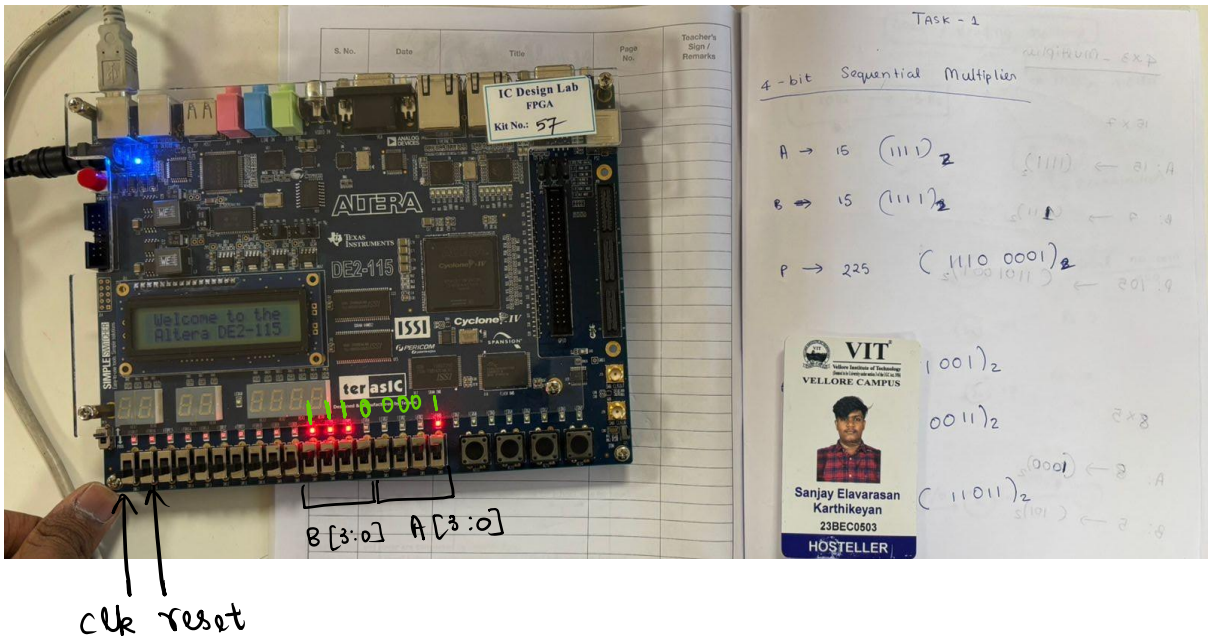
Quartus Prime – FPGA implementation

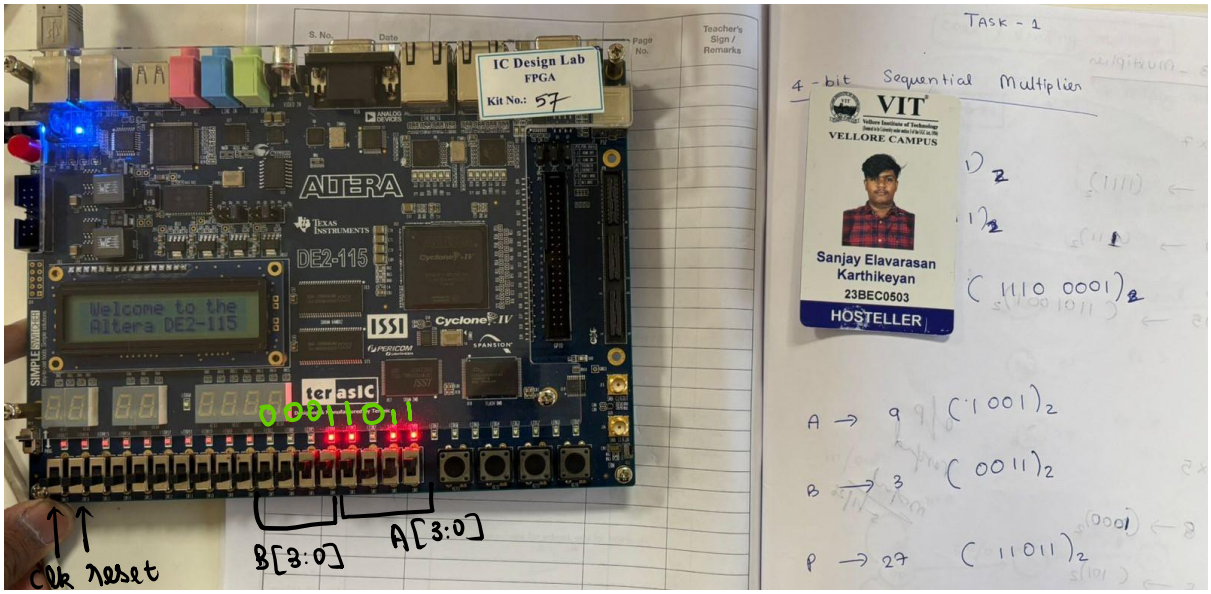


23BEC0503



FPGA OUTPUT :





INFERENCE

The full adder was successfully designed using gate-level modeling and verified through simulation, producing correct sum and carry outputs. Using this full adder, a 4-bit Ripple Carry Adder was implemented and validated, demonstrating proper carry propagation. The RCA was further used to design a 4×3 multiplier, and simulation results confirmed accurate multiplication. The designs were synthesized and implemented on the FPGA, where the hardware outputs matched the simulated results. This experiment demonstrated modular design, verification using testbenches, and successful implementation of arithmetic circuits on FPGA